SE LAB

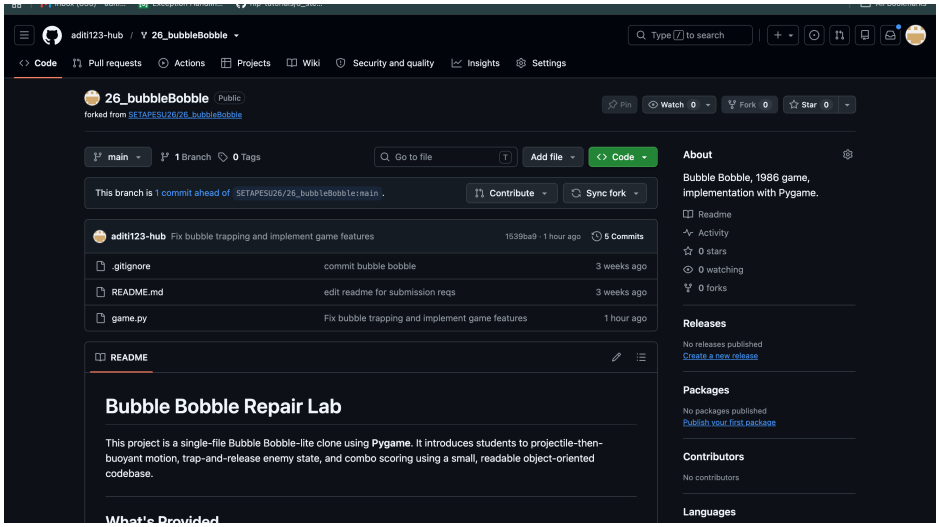
Name: Aditi Agarwal

SRN: PES2UG24CS029

Repo Assigned: Bubble Bobble Repair Lab

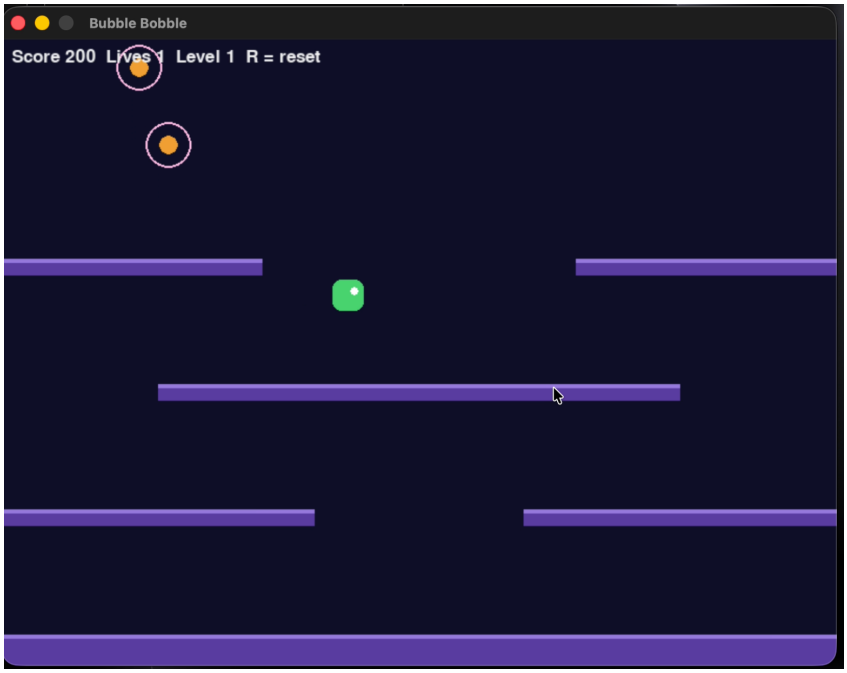
Personal Repo: LAB-4

Forked: 26_bubbleBobble



Original Game

Initial Execution and Basic Testing



Identifying Bugs and Fixing

1. Fixing the enemy-trapping distance check

Prompt:

I'm working on a Bubble Bobble game in Python and found a bug in the `trap_enemies()` function. Bubbles sometimes pass very close to enemies without trapping them.

The README says the issue is with the squared-distance comparison. Can you explain what's wrong, show me the smallest fix, and tell me how to test it? Please keep it simple and don't change any unrelated code.

Here's the function:

```
def trap_enemies(self, bubble):
    if bubble.enemy or bubble.age >= 1.5:
        return
    for enemy in self.enemies:
        if bubble.pos.distance_squared_to(enemy.center) < 24 * 2:
            self.enemies.remove(enemy)
            bubble.enemy, bubble.vel.x = enemy, bubble.vel.x * 0.2
            return
```

The bug is in this line:

```
if bubble.pos.distance_squared_to(enemy.center) < 24 * 2:
```

1. What is wrong?

`distance_squared_to()` returns the square of the distance, but `24 * 2` equals `48`, not \(24^2 = 576\).

So, the bubble only traps an enemy when the squared distance is less than 48, making the detection range much smaller than intended.

2. Smallest fix

Replace just one line:

```
if bubble.pos.distance_squared_to(enemy.center) < 24 ** 2:
```

Why? `24 ** 2` equals 576, so both sides of the comparison now use squared-distance units.

Alternatively, you can write `24 * 24`.

3. How to test it

1. Run the game and move bubbles close to enemies.
2. Check that enemies get trapped when bubbles come within the intended distance of 24 pixels.

3. Verify that enemies outside this range are not trapped.
4. Test again to ensure existing bubble behaviour still works.

Expected result: Bubbles should reliably trap nearby enemies without changing any unrelated code.

```
    def trap_enemies(self, bubble):
        if bubble.enemy or bubble.age >= 1.5:
            return
        for enemy in self.enemies:
            if bubble.pos.distance_squared_to(enemy.center) < 24 ** 2:
                self.enemies.remove(enemy)
                bubble.enemy, bubble.vel.x = enemy, bubble.vel.x * 0.2
                return
```

2. Implementing bubble_tint(bubble)

Prompt:
I'm working on the Bubble Bobble game. The `bubble_tint(bubble)` function is currently empty. Can you explain how to implement it so bubbles change colour as their lifetime runs out? Please show me a simple solution and explain how to test it without changing unrelated code.

You can implement `bubble_tint(bubble)` by changing the bubble's colour based on its age. As the bubble gets older, its colour can gradually change to show that its lifetime is running out.

1. Simple implementation

The exact code depends on how your game represents colours and bubble lifetime. If your game uses Pygame and bubbles have an `age` attribute, you can use this simple example:

```
def bubble_tint(bubble):
    t = min(bubble.age / 1.5, 1.0)

    start_color = (100, 200, 255)
    end_color = (255, 100, 100)

    return tuple(
        int(start + (end - start) * t)
        for start, end in zip(start_color, end_color)
    )
```

2. How it works

- `bubble.age / 1.5` calculates how much of the bubble's 1.5-second lifetime has passed.
- `min(..., 1.0)` ensures the value never exceeds `1.0`.
- `start_color` is the bubble's initial light blue colour.
- `end_color` is its final reddish colour.
- The formula gradually blends the RGB values between these colours.

For example:

Bubble age	Expected colour
0 seconds	Light blue
0.75 seconds	A blend of blue and red
1.5 seconds	Reddish

3. How to test it

1. Run the game and create a bubble.
2. Observe its colour immediately after creation.
3. Watch whether the colour gradually changes as the bubble ages.
4. Confirm that the bubble reaches the final colour near 1.5 seconds.
5. Check that bubbles still trap enemies and disappear according to the existing game logic.

Important: This implementation assumes the function should return an RGB tuple and that the bubble lifetime is 1.5 seconds. Before using it, check how `bubble_tint()` is called in your `game.py` file. If the existing code expects a different return value, adapt the function to that interface rather than changing unrelated code.

```
def bubble_tint(bubble):
    """Return a colour based on the bubble's remaining lifetime."""
    if bubble.enemy is None:
        return (120, 230, 255)

    if bubble.life > 5:
        return (255, 190, 230)
    elif bubble.life > 2:
        return (255, 140, 180)
    else:
        return (255, 70, 70)
```

3. Implementing on_fruit_collected(fruit)

Prompt:
I'm working on the Bubble Bobble game in Python. The `on_fruit_collected(fruit)` function is currently empty. Can you suggest a simple way to show when a player collects fruit? Please explain the code, make the smallest necessary change, and tell me how to test it without changing unrelated parts of the game.

A simple way to show that a player has collected fruit is to display a score message or increase the player's score when the fruit is collected. The smallest change depends on what your game already supports.

1. Simple implementation

If your game already has a score attribute, you could implement the function like this:

```
def on_fruit_collected(self, fruit):
    self.score += 100
```

This increases the player's score by 100 points whenever fruit is collected.

2. Explanation

- `on_fruit_collected(self, fruit)`: Called when the player collects fruit.
- `self.score += 100`: Adds 100 points to the existing score.

Important: This assumes your game already uses `self.score` and initializes it elsewhere. If it uses a different score variable, use that existing variable instead.

3. How to test it

1. Start the game and note the current score.
2. Collect a fruit.
3. Check that the score increases by 100 points.
4. Collect another fruit and verify that the score increases again.
5. Confirm that fruit collection and other game mechanics still work as before.

This is a minimal change that avoids modifying unrelated code.

If your game doesn't have a score system, share the relevant fruit collection code from `game.py`, and I can suggest the smallest change that fits your existing implementation.

```
def on_fruit_collected(fruit):
    """Display a message when fruit is collected."""
    print(f"Fruit collected! +{fruit.value} points")
```

4. Implementing bonus_life_threshold()

Prompt:
I'm working on the Bubble Bobble game in Python. The `bonus_life_threshold()` function is currently empty. The README suggests awarding an extra life every 5,000 points. Can you explain how to implement this with the smallest change and tell me how to test it? Please keep it simple and don't change unrelated code.

The simplest solution is to make `bonus_life_threshold()` return `5000`, so the game knows that the player earns a bonus life for every 5,000 points.

1. Smallest fix

If the function is expected to return the threshold, implement it like this:

```
def bonus_life_threshold(self):
    return 5000
```

2. Explanation

- `return 5000` sets the bonus-life threshold to 5,000 points.
- The existing game logic can use this value to determine when to award an extra life.
- No other code needs to change, provided the game already checks this threshold and awards the life.

3. How to test it

1. Run the game and score points until you reach 5,000.
2. Check that you receive an extra life.
3. Continue scoring until you reach 10,000 and check for another extra life.
4. Verify that no extra life is awarded before the next threshold.

Important: This implementation only supplies the threshold. If the game doesn't already contain logic to award extra lives, returning `5000` alone won't award them.

```
def bonus_life_threshold():
    """Award an extra life for every 5000 points."""
    return 5000
```